

5.9 회귀 실습 - 자전거 대여 수요 예측

1) Intro

-> 2011.01 ~ 2012.12 날짜/시간, 기온, 습도, 풍속 정보 기반 1시간 간격 동안의 자전거 대여 횟수 기재 데이터
-> 주요 칼럼

- datetime: hourly date + timestamp
- season: 1 = 봄, 2 = 여름, 3 = 가을, 4 = 겨울
- holiday: 1 = 토, 일요일의 주말을 제외한 국경일 등의 휴일, 0 = 휴일이 아닌 날
- workingday: 1 = 토, 일요일의 주말 및 휴일이 아닌 주중, 0 = 주말 및 휴일
- weather:
 - 1 = 맑음, 약간 구름 낀 흐림
 - 2 = 안개, 안개 + 흐림
 - 3 = 가벼운 눈, 가벼운 비 + 천둥
 - 4 = 심한 눈/비, 천둥/번개
- temp: 온도(섭씨)
- atemp: 체감온도(섭씨)
- humidity: 상대습도
- windspeed: 풍속
- casual: 사전에 등록되지 않는 사용자가 대여한 횟수
- registered: 사전에 등록된 사용자가 대여한 횟수
- count: 대여 횟수

2) 데이터 클렌징 및 가공과 데이터 시각화

-> 데이터 세트 이용 모델 학습, 대여 횟수(count)예측

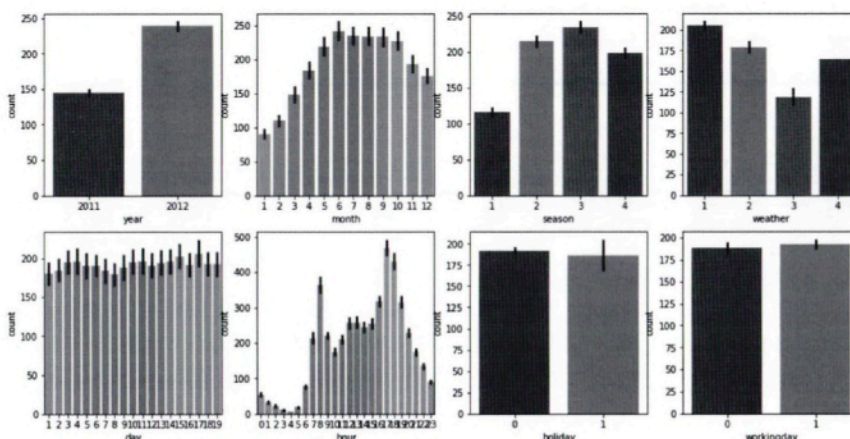
a. bike_df.head() => 10886개 레코드, 12개 칼럼

b. bike_df.info() => 데이터 칼럼 타입 확인

c. 판다스 문자열 datetime 타입 변환 apply 메서드 활용 => 년, 월, 일, 시간 칼럼 추출

d. Datetime, casual, registered 칼럼 삭제

e. 주요 칼럼별로 Target 값 count가 어떻게 분포되어 있는지 시각화 => 시본 barplot



Year: 2012 > 2011
Month: 6, 7, 8, 9 up
Season: 여름, 가을
Weather: 맑음, 안개
Hour: 출퇴근 시간
Day: 차이 약소
휴일 여부: 주중 up

f. 회귀 모델 적용 (RMSLE, MAE, RMSE)

-> Root Mean Square Log Error 평가 함수

-> 주의점: rmsle 구할 때 데이터 값의 크기에 따라 오버플로/언더플로 에러 발생 easy => 넘파이 로그, 사이킷런 메서드 사용에 주의하시라 => log1p() 이용

3) 로그 변환, 피쳐 인코딩과 모델 학습/예측/평가

=> 회귀 모델을 이용해 자전거 대여 횟수 예측 모델 구축

↳ 결괏값이 정규 분포로 되어 있는지 확인, 카테고리형 회귀 모델의 경우 원-핫 인코딩으로 피쳐 인코딩

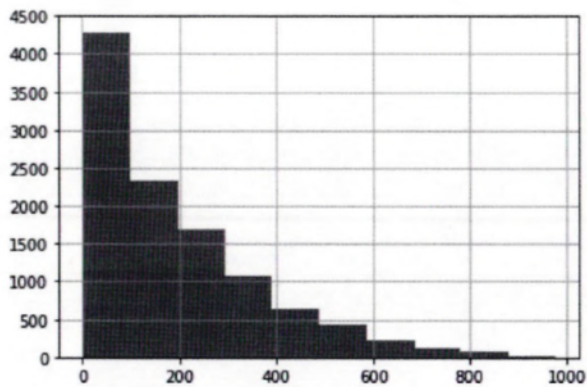
a. Linear Regression 객체 이용 회귀 예측

-> RMSLE : 1.165, RMSE : 140.900, MAE : 105.924 / 예측 오류로 비교적 큰 값 도출

b. df의 프레임으로 만들어 오류값이 가장 큰 순으로 5개 확인

-> 오류값 546 ~ 568 => 예측 오류가 크다! Target 값 분포가 왜곡된 형태인지 확인해보자~

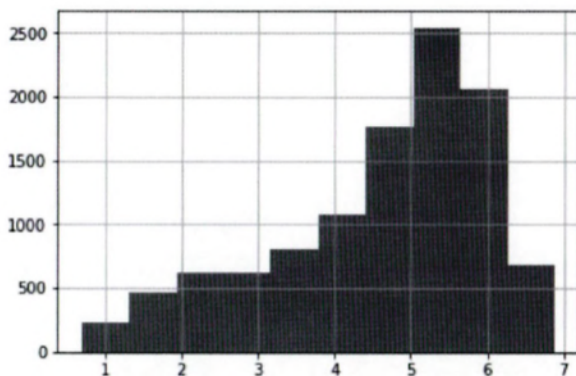
c. hist() 자전거 대여 모델의 count 칼럼이 정규 분포인지 확인



=> 정규 분포 X, 0~200 사이에서 왜곡

=> 로그 적용, 변환!

d. 넘파이 log1p() 이용 -> expm1() 적용, scale값으로 원상 복구

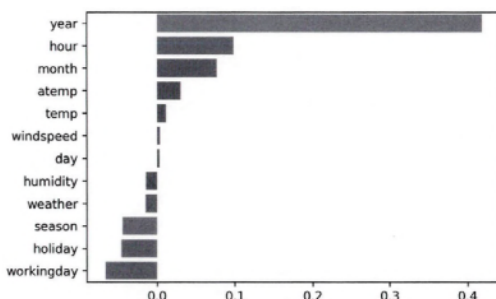


=> 그나마 조금 나아짐

e. 다시 학습, 평가 수행

-> RMSLE : 1.017, RMSE : 162.594, MAE : 109.286 / RMSLE 오류 다운, RMSE up

f. 피쳐 회귀 계숫값 시각화



=> 년도 피처가 개별 숫자값처럼 처리됨... 하지만 이들은 카테고리형 피처임

g. 피쳐 인코딩에 원-핫 인코딩 적용하여 변환

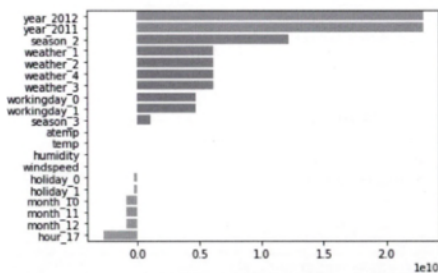
h. LinearRegression, Ridge, Lasso 모두 학습해 예측 성능 확인

-> get_model_predict(): 모델과 학습/테스트 데이터 세트 입력 시 성능 평가 수치 반환 함수 def하기

```
### LinearRegression ###
RMSLE: 0.590, RMSE: 97.688, MAE: 63.382
### Ridge ###
RMSLE: 0.590, RMSE: 98.529, MAE: 63.893
### Lasso ###
RMSLE: 0.635, RMSE: 113.219, MAE: 72.803
```

=> 꽤 많이 향상됨

i. 회귀 계수가 높은 피쳐 다시 시각화 (상위 20개)



=> 피쳐 영향도 변화, 모델 성능 향상

j. 회귀 트리를 이용해 회귀 예측 수행

```
### RandomForestRegressor ###
RMSLE: 0.354, RMSE: 50.356, MAE: 31.120
### GradientBoostingRegressor ###
RMSLE: 0.330, RMSE: 53.335, MAE: 32.744
### XGBRegressor ###
RMSLE: 0.342, RMSE: 51.732, MAE: 31.251
### LGBMRegressor ###
RMSLE: 0.319, RMSE: 47.215, MAE: 29.029
```

=> 랜덤포레스트, GBM, XGBoost, LightGBM 순차 성능 평가

=> 학습/테스트 데이터 넘파이 ndarray 로 변환

=> 선형회귀 모델보다 회귀 예측 성능 개선

5.10 회귀 실습 - 캐글 주택 가격: 고급 회귀 기법

1) Intro

-> house_price.csv: 79개 변수, 미국 아이오와 주 에임스 지방의 주택 가격 정보

2) 데이터 사전 처리

- a. 원본 csv 파일 기반 df 보관, 복사하여 데이터 가공
- b. 데이터 세트 전체 크기, 칼럼 타입, Null 칼럼과 건수 내림차순 출력
=> 1460개 레코드, 81개 피쳐, 1000개 이상 데이터가 null (-> 드롭!)

c. 정규 분포 확인



=> 왼쪽으로 치우친 형태

d. 로그 변환 적용



=> 정규 분포 형태

e. SalePrice 로그 변환 후 DataFrame 반영, Null값 많은 피쳐 드롭

```
## Null 피쳐의 Type :  
MasVnrType      object  
BsmtQual        object  
BsmtCond        object  
BsmtExposure    object  
BsmtFinType1    object  
BsmtFinType2    object  
Electrical      object  
GarageType      object  
GarageFinish    object  
GarageQual      object  
GarageCond      object
```

-> Null값 없음

f. 원-핫 인코딩 변환 / 판다스 get_dummies()

```
get_dummies() 수행 전 데이터 Shape: (1460, 75)
get_dummies() 수행 후 데이터 Shape: (1460, 271)
## Null 피처의 Type :
Series([], dtype: object)
```

=> 피처 271개로 증가, null 값 피처 없음

3) 선형 회귀 모델 학습/예측/평가

a. RMSE 계산 함수 생성

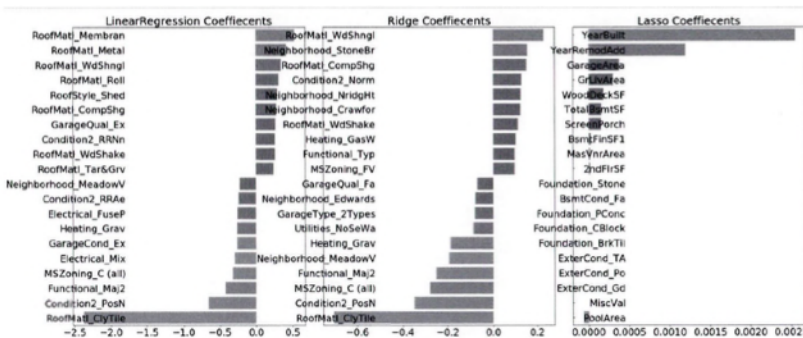
b. 선형 회귀 모델 학습, 예측, 평가

```
LinearRegression 로그 변환된 RMSE: 0.132
Ridge 로그 변환된 RMSE: 0.128
Lasso 로그 변환된 RMSE: 0.176
```

=> 최적 파라미터 튜닝 필요!

c. 회귀 계수 값의 상, 하위 10개 피처명과 그 회귀 계수값을 가지는 판다스 Series 객체 반환 함수 생성

d. 모델별 회귀 계수 시각화



-> 라소의 경우 다른 두 개 모델과 다른 회귀 계수 형태, 데이터 분할에 문제?

f. 전체 데이터 세트를 교차 검증 폴드 세트로 분할, 평균 RMSE 측정 (-> cross_val_score() 이용)

```
Ridge CV RMSE 값 리스트: [0.117 0.154 0.142 0.117 0.189]
Ridge CV 평균 RMSE 값: 0.144
Lasso CV RMSE 값 리스트: [0.161 0.204 0.177 0.181 0.265]
Lasso CV 평균 RMSE 값: 0.198
```

-> 라소의 경우 릿지 모델보다 성능 떨어짐

g. 모델별로 최적화 하이퍼 파라미터 작업 반복 진행 후 별도 함수 생성

```
Ridge 5 CV 시 최적 평균 RMSE 값: 0.1418, 최적 alpha: {'alpha': 12}
Lasso 5 CV 시 최적 평균 RMSE 값: 0.142, 최적 alpha: {'alpha': 0.001}
```

-> 라소 모델 성능 좋아짐

h. 선형 모델에 최적 alpha 값 설정, 분할된 학습/테스트 데이터 이용하여 학습/예측/평가 수행 후 모델별 회귀 계수 시각화



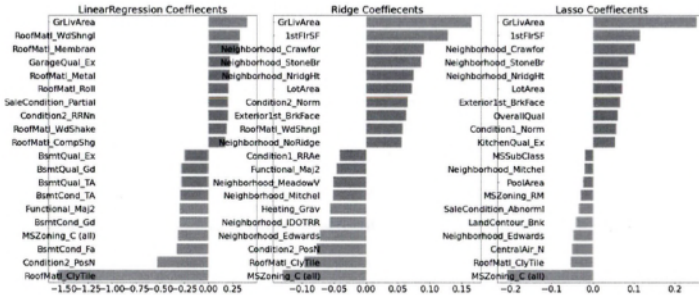
-> 릿지와 라소 모델에서 비슷한 피처의 회귀 계수 높음

-> 라소의 경우 동일한 피처라도 회귀 계수 값이 작음

- i. DataFrame 의 숫자형 피처의 왜곡 정도 확인
- j. 왜곡 정도가 높은 피처 로그 변환
- k. 원-핫인코딩을 적용한 house_df_ohe 구축

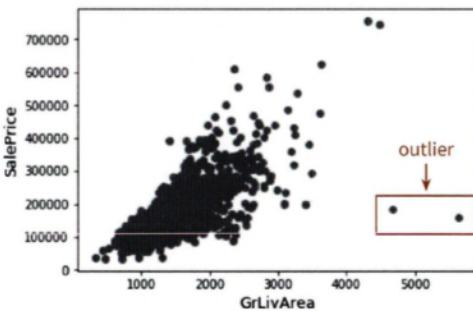
Ridge 5 CV 시 최적 평균 RMSE 값:0.1275, 최적 alpha: {'alpha': 10}
 Lasso 5 CV 시 최적 평균 RMSE 값:0.1252, 최적 alpha: {'alpha': 0.001}

-> 평균 RMSE 값 향상



-> 주거 공간의 크기가 주택 가격에 미치는 영향이 높을 것이라는 결과 도출

l. GrLivArea와 SalePrice 관계 시각화



-> 해당 피처가 SalePrice와 양의 상관도가 높음

m. 대상 데이터 필터링

이상치 레코드 index : [523 1298]
 이상치 삭제 전 house_df_ohe shape: (1460, 271)
 이상치 삭제 후 house_df_ohe shape: (1458, 271)

-> 전체 레코드 감소

n. 피처/타겟 데이터 세트 다시 생성 후 최적화 수행, 결과 출력

Ridge 5 CV 시 최적 평균 RMSE 값: 0.1125, 최적 alpha: {'alpha': 8}
 Lasso 5 CV 시 최적 평균 RMSE 값: 0.1122, 최적 alpha: {'alpha': 0.001}

-> 이상치 데이터 제거 후 예측 수치 크게 향상

4) 회귀 트리 모델 학습/예측/평가

a. 하이퍼 파라미터 설정 미리 적용 후 평균 RMSE 값 (XGBoost)

XGBRegressor 5 CV 시 최적 평균 RMSE 값: 0.1178, 최적 alpha: {'n_estimators': 1000}

-> 하이퍼 파라미터 설정 미리 적용 후 평균 RMSE 값

b. 하이퍼 파라미터 설정 미리 적용 후 평균 RMSE 값 (LightGBM)

-> LGBMRegressor 5 CV 시 최적 평균 RMSE 값: 0.1163, 최적 alpha : {'n_estimators' : 1000}

5) 회귀 모델의 예측 결과 혼합을 통한 최종 예측

a. 최종 혼합 모델, 개별 모델 RMSE 값을 출력하는 함수 생성 후 각 모델의 예측값 계산하여 각각 RMSE 도출

```
최종 혼합 모델의 RMSE: 0.10007930884470506  
Ridge 모델의 RMSE: 0.10345177546603253  
Lasso 모델의 RMSE: 0.10024170460890032
```

-> 최종 모델이 개별 모델보다 성능 개선

b. XGBoost + LightGBM

```
최종 혼합 모델의 RMSE: 0.10170077353447762  
XGBM 모델의 RMSE: 0.10738295638346222  
LGBM 모델의 RMSE: 0.10382510019327311
```

-> 최종 모델이 개별 모델보다 성능 개선

6) 스택킹 앙상블 모델을 통한 회귀 예측

a. `get_stacking_base_datasets()`

-> 인자로 개별 기반 모델, 원래 학습/테스트 피쳐 데이터 입력

-> 함수 내부에서 개별 모델이 K-폴드 세트에 설정된 폴드 세트 내부에서 원본 학습 데이터 재추출 후 학습/예측 수행하여 결과 저장

b. `get_stacking_base_datasets()` 모델별로 적용

c. 개별 모델이 반환하는 학습용 피쳐 데이터와 테스트용 피쳐 데이터 세트 결합 후 최종 메타 모델에 적용

-> 스택킹 회귀 모델의 최종 RMSE 값은 : 0.09799152965189684

-> 현재까지 가장 좋은 성능 평가

- 2) 회귀 트리 모델 학습/예측/평가
- 3) 회귀 모델의 예측 결과 혼합을 통한 최종 예측
- 4) 스택킹 앙상블 모델을 통한 회귀 예측